

Ejercicio 1

Crea un sistema de gestión económica para una empresa. El sistema tiene que implementar las siguientes clases e interfaces:

Clase abstracta ApunteEconomico
int code es un código numérico compuesto por el año seguido de un número serial único
double importe: el importe asociado con este apunte
String concepto: una descripción de este apunte.
getter y setter
int getEjercicioActual() devuelve el año de este apunte
abstract void describe() imprime en pantalla la descripción de este ApunteEconomico

Clase Factura hereda de ApunteEconomico
int idFactura identificador de esta factura
String emisor
getter y setter

Clase GastoEmpresarial hereda de ApunteEconomico
String fecha
String DNIResponsable
getter y setter

La clase GestionEconomica implementa el main.

GestionEconomica almacena los ApuntesEconomicos en un HashMap cuyas claves son los code de los apuntesEconomicos y los valores son objetos ApunteEconomico. En la clase que GestionEconomica se declararán métodos para realizar las siguientes funcionalidades:

- Crear un método "crearApunte" que permita añadir una Factura o un GastoEmpresarial pasados todos sus datos por parámetro. Si al insertar datos el usuario se equivoca en insertar el tipo de dato correcto, el método indicará "Operación anulada" y terminará.
- Crear un método "buscaApunte" que dado un code busca el apunteEconomico con ese code. En caso de no encontrarse, el método lanzará una excepción ApunteNoEncontradoException. GestionEconomica no termina si salta esta excepción.
- Crear un método "importeTotalFacturas" que recibe un año como parámetro e indica el importe total de las Facturas de ese determinado ejercicio (año). Si no hay ningún apunteEconomico de ese año, lanzará la excepción EjercicioNoEncontradoException. GestionEconomica no termina si salta esta excepción
- La clase GestionEconomica tendrá, además de los métodos solicitados, el método main que realizará lo siguiente:
 - o Crear dos apuntes
 - o Buscar un apunte solicitando un código previamente
 - o Mostrar el importe total de las facturas solicitado un año previamente

	RA4	RA6	RA7
Implementar crearApunte	1.5	1	2
Implementar buscaApunte		1	1.5
Implementar importeTotal	1.5	3	1.5
Implementación correcta de las clases	2		

Ejercicio 2

Realizar una pequeña aplicación en Java para gestionar los abonados de un gimnasio. El diseño de clases será el siguiente:

- Interfaz **Abonado**, que incluya los métodos boolean registrarActividad(), boolean cancelarActividad(String nombre)
- Dos implementaciones de Abonado: **AbonadoRegular** y **AbonadoVIP**. Todos los abonados se caracterizan por su nif, nombre y conjunto de actividades en las que están registrados (debe ser una colección tipo Set). La diferencia entre ambos tipos de abonado es que los abonados de tipo regular tienen un máximo de actividades, y los abonados de tipo VIP no tienen límite.
Este valor máximo de actividades para los abonados regulares se solicitará al comienzo de la aplicación.

En el registro de la actividad, en el caso de los abonados regulares, solo se registrará si no se ha llegado al límite máximo. En ambos casos el método devolverá true si se pudo registrar la actividad correctamente (porque no existía previamente o porque no se había alcanzado el máximo), y false en caso contrario.

Los datos de la actividad se solicitarán por teclado.

Cuando se cancela la actividad, se devolverá true si existía y se eliminó y false en caso de que no existiera.

- Clase **Actividad**: se caracteriza por el nombre, la cuota, y un array de caracteres que define los días a la semana en los que se realiza ('L','M','X',...).
- Programa principal: en primer lugar configurará el número máximo de actividades para los usuarios regulares (guardando el dato donde el alumno estime oportuno), y luego mostrará un menú con las opciones:
 - Insertar abonado (sin actividades): los abonados se incorporarán a una lista
 - Registrar actividad para un abonado dado su nif
 - Mostrar el dinero que se ingresa por todas las actividades de los abonados regulares
 - Mostrar un listado con los nombres de los abonados que tienen alguna actividad un día de semana solicitado por teclado separados por Abonados normales y Abonados Vip.

	RA4	RA6	RA7
Implementación clases e interfaz	1		
Implementación de insertar abonado	1	2	
Implementación de registrar actividad	1	1	
Implementación de mostrar dinero	1	1	3
Implementación de listado por día de la semana	1	1	2

NOTA: si el alumno obtiene al menos 5 puntos en cada RA evaluado en este examen (4, 6 y 7), se considerarán recuperados también (con calificación 5) los RA 1, 2 y 3.